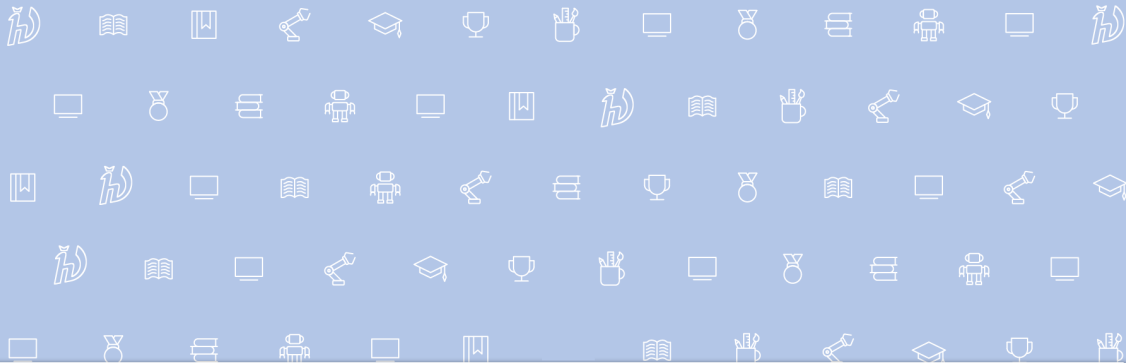




Artificial Intelligence Software, Lecture 10

Web APIs and AI Interfaces: REST, FastAPI, and Gradio



Center for
Cognitive
Modeling



Lecture Plan

1. **TCP/IP and HTTP** — how the web works under the hood
2. **REST API** — design principles and HTTP methods
3. **FastAPI** — building REST servers in Python
4. **Gradio** — instant web UIs for AI models
5. **Putting it together** — exposing a model via FastAPI + Gradio

Extra material (Jupyter notebooks):

`extra_material/rest_api_example.ipynb`

`extra_material/gradio_example.ipynb`

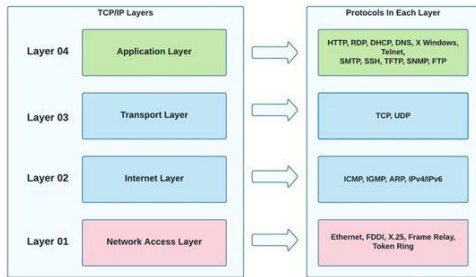


The Internet in Layers

TCP/IP stack overview

1. **Link layer** — hardware frame delivery within one network segment (Ethernet, Wi-Fi)
2. **Internet layer** — packet routing across networks (IP)
3. **Transport layer** — reliable data delivery between hosts (TCP, UDP)
4. **Application layer** — structured data exchange between programs (**HTTP**, FTP, DNS...)

TCP/IP Model



We work at the **Application layer**.

Everything below is handled by the OS and hardware.



HTTP Request and Response

The language of the web

Request structure

- **Request line:** method + URL + version
- **Headers:** metadata (Host, User-Agent, Accept, ...)
- **Empty line**
- **Body:** optional data (JSON, form fields, file)

HTTP methods

GET • POST • PUT • PATCH • DELETE

Response structure

- **Status line:** version + **status code** + message
- **Headers:** Content-Type, Content-Length, ...
- **Empty line**
- **Body:** HTML, JSON, binary data, ...

Status code families

2xx success • 3xx redirect

4xx client error • 5xx server error



Query Strings and URL Encoding

Parameters are appended after ? and separated by &:

`https://api.example.com/search?term=hello&limit=10`

Only ASCII letters, digits, and a few symbols are safe. Everything else must be **percent-encoded**:

Character	Encoded
Space	%20
&	%26
Non-ASCII (e.g. ü)	%C3%BC

https://wikireality.ru/wiki/%D0%9E%D0%BB%D0%B5%D0%B3|

Олер — Викиреальность - <https://wikireality.ru/wiki/Олер> [Переключиться на эту вкладку](#)

https://wikireality.ru/wiki/%D0%9E%D0%BB%D0%B5%D0%B3 - Поиск Яндекс

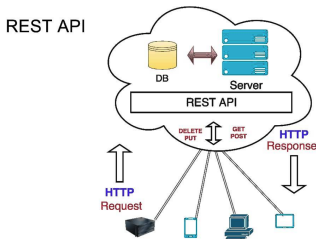


What Is a REST API?

Representational State Transfer

A **REST API** is an architectural style for building web services:

- **Resources** are data objects (user, post, task, ...) identified by URIs
- **Standard HTTP methods** define the operation
- **Stateless**: every request is self-contained, no server session
- Data exchanged as **JSON** (typically)



Method	Action	CRUD
GET	Retrieve resource	Read
POST	Create resource	Create
PUT	Replace resource	Update
PATCH	Partial update	Update
DELETE	Remove resource	Delete



REST API — Tasklist Example

Mapping HTTP routes to CRUD operations

Route	Method	Action
/tasks	GET	Return list of all tasks
/tasks	POST	Create a new task
/tasks/{id}	GET	Return a single task by ID
/tasks/{id}	PUT	Replace the task with new data
/tasks/{id}	PATCH	Update selected fields of the task
/tasks/{id}	DELETE	Delete the task

URL identifies *what* resource; HTTP method identifies *what to do* with it.



Python requests Library

Making HTTP requests from Python

Key features

- One function per HTTP method: `requests.get()`, `.post()`, `.put()`, `.delete()`, ...
- Response object: `resp.status_code`, `resp.headers`, `resp.json()`, `resp.content`
- Raise exception on bad status: `resp.raise_for_status()`
- `requests.Session()` — reuses TCP connection, shares headers/cookies
- Streaming downloads: `stream=True` + `iter_content()`

Testing endpoints

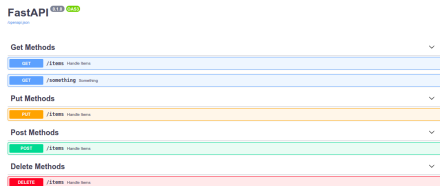
- httpbin.org — echoes your request back
- jsonplaceholder.typicode.com — fake REST data



What Is FastAPI?

FastAPI is a Python web framework for building REST APIs quickly:

- **Decorator-based routing:** `@app.get("/items")`
- **Automatic data validation** via Pydantic models
- **Auto-generated docs:** Swagger UI at `/docs`
- **Async support** out of the box (`async def`)
- Among the **fastest** Python frameworks (comparable to Node.js)



FastAPI = **Flask-like simplicity** + **async performance** + **automatic OpenAPI docs**



Building a REST Server with FastAPI

Core concepts

Application structure

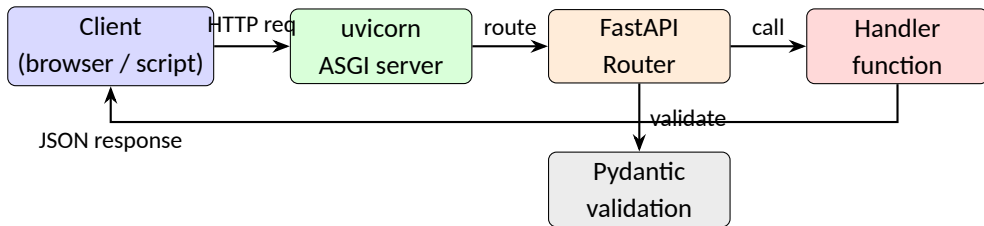
- Create an `FastAPI()` app instance
- Define **route handlers** with decorators
- Use **path parameters** (`/items/{id}`) and **query parameters**
- Use **Pydantic models** to declare request/response schemas
- Run with `uvicorn app:app --reload`

Request body vs. query string

- Query string: simple scalars in the URL
- Request body: structured JSON (POST/PUT/PATCH)

FastAPI — Request Lifecycle

From HTTP bytes to Python and back



- **uvicorn** — ASGI web server (like gunicorn for WSGI)
- **Pydantic** — validates and coerces input/output data types automatically
- **OpenAPI schema** generated automatically from type annotations



FastAPI vs. Other Frameworks

When to choose what

	Flask	Django REST	FastAPI
Paradigm	Sync (WSGI)	Sync (WSGI)	Async (ASGI)
Speed	Moderate	Moderate	High
Data validation	Manual	DRF serializers	Pydantic (auto)
API docs	None built-in	DRF browsable API	Swagger + ReDoc
Learning curve	Low	Medium	Low-Medium
Best for	Small services	Full Django projects	Modern APIs, ML serving

For AI/ML model serving FastAPI is the de-facto standard: lightweight, fast, and self-documenting.



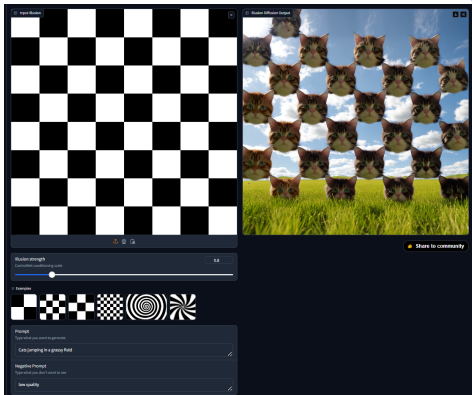
What Is Gradio?

Gradio is a Python library that turns any function into a shareable web app:

- Define your function
- Declare inputs and outputs as **components**
- Call `demo.launch()` — that is all

Under the hood

- Gradio starts a local **FastAPI** server
- Your function is exposed as a REST endpoint
- A prebuilt React frontend
- `share=True` creates a public tunnel via





Gradio Components Cheatsheet

Inputs and outputs for every modality

Text & numbers

- `gr.Textbox()` — plain / multi-line text
- `gr.Number()` — numeric field
- `gr.Slider(min, max)` — range picker

Choice

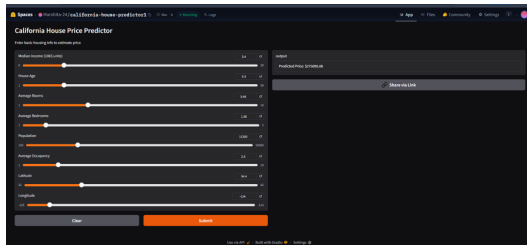
- `gr.Radio(choices=[...])`
- `gr.Dropdown(choices=[...])`
- `gr.Checkbox()`

Media

- `gr.Image(type="pil")`
- `gr.Audio()`, `gr.Video()`

Display

- `gr.Label()` — classification result
- `gr.Dataframe()` — table





gr.Interface **VS.** gr.Blocks

Two levels of layout control

gr.Interface — simple wrapper

- Pass fn, inputs, outputs
- Generates a standard two-column layout automatically
- Good for quick demos and single-function apps

gr.Blocks — full control

- Place components in rows/columns manually
- Bind **multiple events**: `.click()`, `.change()`, `.submit()`
- Multiple inputs/outputs per event
- Multi-step or multi-model pipelines

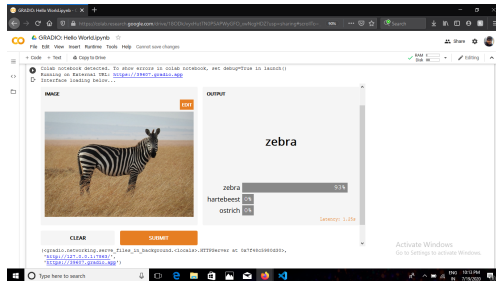
Gradio + HuggingFace Models

Pattern

1. Load model with `transformers.pipeline(...)`
2. Write a thin wrapper function
3. Wrap with `gr.Interface(fn=wrapper, ...)`
4. Call `.launch()`

Available tasks out of the box

- Sentiment analysis, text generation
- Image classification, object detection
- Speech recognition, translation





Putting It Together

Model serving architecture — two approaches

Approach A: FastAPI only

- POST /predict accepts JSON, returns JSON
- Frontend is separate (React, mobile app, ...)
- Best for production APIs consumed by other services

Approach B: Gradio only

- One Python file, instant UI + API
- Built-in REST endpoint at /api/predict
- Best for demos, research, rapid prototyping

Approach C: FastAPI + Gradio mounted together

- Gradio app mounted into FastAPI via `app.mount()`
- One server, both UI and API routes



Summary

- **TCP/IP & HTTP:** how clients and servers talk — request/response, methods, status codes, headers
- **REST API:** uniform interface for resources using HTTP — GET/POST/PUT/PATCH/DELETE mapped to CRUD
- **Python requests:** consuming REST APIs from Python
- **FastAPI:** building REST servers — routing, Pydantic validation, auto-docs, async
- **Gradio:** wrapping any Python function in a web UI — components, Interface vs. Blocks, HuggingFace integration

Extra material (notebooks)

- `rest_api_example.ipynb`
TCP/IP stack, HTTP deep dive, requests library with GET/POST/PUT/PATCH/DELETE examples
- `gradio_example.ipynb`
Hello World, components, Blocks, model integration (sentiment, image classification)



References

- Fast API Tutorial (playlist)
- What is REST API?
- Что такое REST API?
- LLM app based on Gradio

Deserve "A" grade!

– Oleg Bulichev

✉ bulichev.ov@mipt.ru

📍 @Lupasic

🏠 Цифра, 521